

PROJECT QUANT — GVM DEVELOPMENT MEMORY

G-Series | Build Journey G1 to G9 | G10 Agenda Locked
Arpit Goel + Claude | April–May 2026

CORE MISSION

93% of retail investors in India lose money — not because they lack intelligence, but because they lack the tools institutions have. Project Quant exists to change that.

Build AI Aladdin — India's first institutional-grade stock intelligence platform for retail investors. A platform where a 25-year-old investor in Lucknow has the same analytical edge as a fund manager in Mumbai.

BUILD STATUS — G1 TO G9

Session	Date	Key Achievement	Status
G1	Apr 2026	Project vision locked. Tech stack decided. Python + VS Code install planned.	Done
G2	Apr 2026	Python 3.14 + VS Code + Pandas installed. gvm_scorer.py created.	Done
G3	Apr 2026	gvm_engine.py built. screener_loader.py reading CSV. 21 parameters mapped.	Done
G4	30 Apr 2026	6 logic fixes applied. 1,806 stocks scored. 127 segments target set.	Done
G5	7 May 2026	3,371 merge issue found. PostgreSQL setup next. Duplicate fix pending.	Done
G6	19 May 2026	Platform architecture expanded. Internal EXE track added. API Library defined.	Done
G7	20 May 2026	Railway-first architecture locked. No MCP. No Cowork. No Desktop dependency.	Done
G8	21 May 2026 AM	Input 2 cleaned. 126 segments locked. Analytics merged. GitHub live.	Done
G9	21 May 2026 PM	PostgreSQL live. FastAPI deployed. 25 endpoints live on Railway.	Done

G9 — TODAY'S ACHIEVEMENTS (21 May 2026)

#	Task	Result
1	Railway CLI installation (npm install -g @railway/cli)	Done — v4.10.x installed
2	Railway login + project link to trade-quant	Done — linked to quant_project service
3	PostgreSQL on Railway (postgres-volume-CQk0)	Done — Online, DATABASE_URL captured
4	FastAPI deployment via `railway up`	Done — 25 endpoints live
5	requirements.txt fix (pandas 2.2.3 + pydantic 2.9.2 for Python 3.13)	Done — wheel install passed
6	Environment variables (DATABASE_URL + PORT) set on Railway	Done — applied to quant_project
7	API name updated: 'GVM Stock Rating' → 'Project Quant — Trading API'	Done — live in title + root
8	Code committed and pushed to GitHub	Done — AGQuant/quant_project private

Key Decisions Locked in G9

- **Deployment via Railway CLI** — GitHub auto-deploy blocked (Railway GitHub App not installed). CLI deploy (`railway up`) bypasses this. Working solution; GitHub App can be installed later.
- **Production URL** — <https://quantproject-production.up.railway.app> — 25 endpoints live, docs at /docs (Swagger UI).
- **Database URL stored as ENV variable** — never in code, never in GitHub. .gitignore protected.
- **API name finalized** — 'Project Quant — Trading API' across title, description, and root response.
- **Python 3.13 compatibility** — pydantic upgraded to 2.9.2, pandas to 2.2.3 (pre-built wheels). pydantic 2.7 + pandas 2.2.0 builds failed because pydantic-core 2.18 doesn't support Python 3.13.

TECH STACK — LIVE

Component	Technology	Status
Backend	Python + FastAPI	Live on Railway
Frontend	React	Week 3
Database	PostgreSQL on Railway	Live (postgres-volume-CQk0)
Hosting Backend	Railway only — no AWS	Live
Hosting Frontend	Vercel (free tier)	Pending Week 3
Data Source	Screener.in CSV + Input 2 sheet	Live
Historical + Price Data	Yahoo Finance (yahoo_hist.py + yahoo_quote.py)	Live — no token
AI Layer	Claude API	Week 4
Version Control	GitHub — AGQuant/quant_project	Live (G8)
Sheet Reading	gsread + service account	Installed
Deploy Tool	Railway CLI (npm @railway/cli)	Live (G9)
MCP	REMOVED — not needed	Deleted
Fyers API	REMOVED permanently	Deleted
AWS	NOT USED — Railway only through Year 1	Decided

RAILWAY ARCHITECTURE — POST G9

Production URL: https://quantproject-production.up.railway.app

DB URL: postgres.railway.internal:5432/railway (private, env var only)

Region: Southeast Asia (Singapore)

Service ID: 3fdc41de-3e9d-485d-b699-a35945e617b9

Layer	What it does	Status
PostgreSQL on Railway	Stores all GVM scores, price history, OHLC, signals	Live (G9)
FastAPI on Railway	25 endpoints — Growth, Reliability, Value, Momentum, Composite	Live (G9)
Browser Claude	Fetches Railway URLs — reads live data anywhere	Active
VS Code + Railway CLI	`railway up` to deploy. No GitHub auto-deploy needed.	Active
Google Sheets	Source data via gsread — live prices only	Active
GitHub	Version control — AGQuant/quant_project (private)	Live (G8)

G10 AGENDA — NEXT SESSION

Foundation phase complete. G10 builds the raw data layer that powers Momentum scoring and daily GVM regeneration. Without raw_prices populated, 5 Momentum APIs return blank.

Phase A — Database Foundation

#	Task	Detail
1	Create 4 PostgreSQL tables	gvm_scores, raw_prices, signals, users
2	Insert Day 1 gvm_analytics.csv	1,757 stocks → gvm_scores table

Phase B — Yahoo 5Y Raw Data Layer

#	Task	Detail
3	Bulk fetch 5Y daily OHLC	1,971 stocks + Nifty50 + BankNifty via yahoo_hist.py
4	Store all OHLC in raw_prices table	Date, Open, High, Low, Close, Volume per stock per day
5	Daily OHLC update scheduler	3:45 PM IST, Mon-Fri only — post-market

Phase C — Validate

#	Task	Detail
6	Test /api/score/gvm-score endpoint	End-to-end from browser via Railway URL
7	Confirm Momentum params computable	DMA50, DMA200, 1Y/3Y returns, 52W vs Index
8	Trigger daily GVM regeneration test	All 1,757 stocks rescored, date-stamped

Why G10 Matters

Momentum scoring (5 of 21 GVM params) needs historical daily OHLC. Without raw_prices populated with 5Y data: DMA50/DMA200 can't compute, 1Y/3Y returns blank, 52W vs Index blank. G10 unlocks Momentum scoring and enables daily GVM regeneration — the foundation for live ratings on the platform.

2 MODELS ONLY — FINAL DECISION

Pivot decision May 20, 2026: Build 2 models via chat only. Quality over speed. Depth over breadth.

#	Model	Why	Priority Timeline
1	GVM Rating Engine	Core product — 95% built post G9	Month 1-2
2	V5 Signal Engine	Live trading — real P&L impact daily	Month 2-4

All other models (Portfolio Health, Screener, Alerts, Pair Trading) — parked indefinitely.

MASTER BUILD SEQUENCE — UPDATED POST G9

Phase 1 — Foundation (G8 COMPLETED)

#	Task	File	Status
1	Fix duplicate merge	screener_loader.py	Done
2	Fix segment count to 126	input.csv + loader	Done
3	Fix Stylam, PFC, KN Agri segments	input.csv	Done
4	Validate 5 stocks	gvm_engine.py	Done
5	Merge analytics + commentary	gvm_analytics.py	Done
6	GitHub setup + first commit	AGQuant/quant_project	Done

Phase 2 — Database & Deploy (G9 COMPLETED)

#	Task	Detail	Status
7	PostgreSQL on Railway	Online, DATABASE_URL captured	Done
8	FastAPI deploy on Railway	25 endpoints live	Done
9	Production URL live	quantproject-production.up.railway.app	Done
10	API name updated	Project Quant — Trading API	Done

Phase 3 — Raw Data Layer (G10 — NEXT)

#	Task	Detail
11	Create 4 DB tables	gvm_scores, raw_prices, signals, users
12	Insert Day 1 gvm_analytics.csv	1,757 stocks scored — first day stored
13	Download 5Y OHLC	1,971 stocks + Nifty50 + BankNifty via yahoo_hist.py
14	Store all OHLC in raw_prices	Foundation for Momentum scoring
15	Daily OHLC update scheduler	3:45 PM IST, Mon-Fri only
16	Momentum parameters via FastAPI	1Y/3Y Return, DMA50, DMA200, 52W vs Index
17	Daily GVM score regeneration	All 1,757 stocks rescored daily, date-stamped

Phase 4 — Frontend (Week 3)

React on Vercel. Search bar → stock name → GVM rating + breakdown + commentary punchline. Backend hits Railway FastAPI.

Phase 5 — AI Layer (Week 4)

Claude API integrated for stock explanation. Plain English commentary on top of GVM scores.

GVM MODEL ARCHITECTURE — 21 PARAMETERS

G — Growth (14 Parameters)

Operational (9): 5Y Sales Growth, 3Y Sales Growth, 5Y Profit Growth, 3Y Profit Growth, QoQ Sales Growth, QoQ Profit Growth, OPM, OPM Expansion, Fixed Asset Growth
Reliability (5): Promoter Holding, Institutional Holding Change, ROCE, Interest Coverage (skip BFSI), Dividend Yield

V — Value (2 Parameters)

PE vs Peer Group (inverted logic — lower PE = higher score) | Potential Annual Upside

M — Momentum (5 Parameters)

1Y Return, 3Y Return, DMA50, DMA200, 52-Week vs Index — ALL computed from Yahoo raw OHLC stored in Railway raw_prices table (G10 build)

Scoring Logic

2-Factor System: Factor 1 (absolute performance) + Factor 2 (relative vs peer group average), averaged. Scale: 2.5 / 5 / 7.5 / 10. Absolute thresholds: above 15% = 10, 5-15% = 7.5, 0-5% = 5, below 0% = 2.5. PE uses inverted logic. Peer averages filtered to exclude outliers (10-90 percentile trim). GVM = G + V + M averaged.

ANALYTICS LAYER — gvm_analytics.py

Single script generates scoring + commentary in one pass. Replaces gvm_scorer.py and commentary_generator.py.

Score Band	Label	Verdict
≥ 8.0	Excellent	Strong Buy
7.0 – 8.0	Good	Buy
6.0 – 7.0	Average	Accumulate
5.0 – 6.0	Below Average	Wait & Watch
< 5.0	Poor	Avoid

Day 1 Verdict Distribution (1,757 stocks)

Verdict	Count	% of Universe
Strong Buy	69	3.9%
Buy	390	22.2%
Accumulate	608	34.6%
Wait & Watch	532	30.3%
Avoid	158	9.0%

QUANT_PROJECT FOLDER — POST G9 STATE

File	Purpose	Status
gvm_engine.py	Core scoring logic — all 21 parameters	v3 — D5 fixes applied
screener_loader.py	Reads CSV, cleans, merges, calls engine	v3 — 126 segments stable
gvm_analytics.py	Scoring + commentary in one pass	G8 deliverable
input.csv	Master universe — 1,971 stocks, 126 final segments	G8 corrected
screener.csv	Screener.in export — 1,922 stocks (Above 500 Cr)	Latest
main.py	FastAPI app — 25 endpoints, name = Project Quant — Trading API	Live on Railway (G9)
yahoo_hist.py	Yahoo historical OHLC	Live — no token
yahoo_quote.py	Yahoo live quote (15min delay)	Live
yahoo_mcp.js	Legacy MCP — kept as archive	Archived
Procfile	Railway deploy config	Live
requirements.txt	Python dependencies (Python 3.13 compatible)	Updated G9
service_account.json	Google Sheets auth	Local only (.gitignored)
.gitignore	Git exclusions	G8 created

GROUND RULES — NON-NEGOTIABLE

- **Input 2 = Master Universe.** 1,971 NSE codes. Output = 1,757 (only stocks with screener match). 214 unmatched stocks ignored from DB entirely.
- **No default-5 padding.** If we don't have data, we don't fake a rating. DB = truth.
- **Railway-first, Railway-only.** No AWS through Year 1.
- **No MCP ever again.** FastAPI on Railway replaces MCP permanently.
- **No Fyers ever again.** Yahoo Finance handles all price data — free, no token.
- **Momentum from Yahoo raw data.** DMA50, DMA200, returns — all computed from stored OHLC (G10).
- **Daily OHLC update.** Post-market scheduler — 3:45 PM IST, Mon-Fri only.
- **Blank values = 5.** Missing data defaults to neutral score (engine logic, not padding).
- **Interest Coverage — skip BFSI.** Banks/NBFCs excluded from this parameter.
- **2 models only.** GVM + V5. All others parked indefinitely.
- **service_account.json never enters GitHub.** .gitignore protects it.
- **DATABASE_URL never in code.** Stored as Railway env variable only.
- **Learning rule.** Explain WHY before HOW. Arpit directs. Claude executes.
- **Deploy method.** `railway up` from VS Code terminal. CLI deploy is canonical.

G10 — NEXT SESSION COMMAND

Create 4 PostgreSQL tables → Insert Day 1 gvm_analytics.csv (1,757 stocks) → Bulk fetch Yahoo 5Y OHLC for 1,971 stocks + Nifty50 + BankNifty → Store in raw_prices → Daily scheduler at 3:45 PM IST → Test /api/score/gvm-score from browser. Foundation for Momentum scoring (DMA50, DMA200, 1Y/3Y returns, 52W vs Index).